

Reviewer Pack — Minimal Kähler Manifold Volume and Communication Complexity ...

Entry 32d29fb2f820 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 07:38:57 UTC

Minimal Kähler Manifold Volume and Communication Complexity Rank Variance

Entry ID: 32d29fb2f820 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 07:38:57 UTC

1. Conjecture

Field A (mathematical branch): Kähler Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

For every instance of a communication complexity problem with matrix rank variance v , the volume of the associated minimal Kähler manifold is upper-bounded by $O(v^{1/3})$ and lower-bounded by $\Omega((v/(n!))^{1/3})$, where n is the number of bits in the input.

Rationale (proposer's reasoning):

The geometric structure of a Kähler manifold could provide insights into the complexity of communication protocols. A connection between the volume of such manifolds and the variance of communication complexity ranks might reveal a deep relationship between geometry and computational efficiency.

Taxonomy category: Kahler Geometry × Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f4f7dc3c2e1a0253

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given communication complexity problem instance with matrix rank variance v , if the computed minimal Kähler manifold volume falls within the bounds $O(v^{1/3})$ and $\Omega((v/(n!))^{1/3})$, where n is the number of bits in the input, then support for the conjecture is provided. If any computed volume exceeds $O(v^{1/3})$ or falls below $\Omega((v/(n!))^{1/3})$ for any seed, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_rank(A):
        rank = 0
        A = gaussian_elimination(A)
        for row in A:
            if any(row):
                rank += 1
        return rank

    def factorial(n):
        result = 1
        for i in range(2, n+1):
            result *= i
        return result

    def kahler_manifold_volume(v, n):
        lower_bound = (v / factorial(n)) ** (1/3)
        upper_bound = v ** (1/3)
```

```

    return lower_bound, upper_bound

n_bits = random.randint(5, 30)
n = 2 ** n_bits
A = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]

rank_variance = matrix_rank(A)

lower_bound, upper_bound = kahler_manifold_volume(rank_variance, n_bits)

if not (lower_bound <= rank_variance <= upper_bound):
    return {
        "metric_name": "Kähler Manifold Volume",
        "metric_value": rank_variance,
        "instances_tested": 1,
        "n_max": n_bits,
        "conjecture_holds": False,
        "counterexample": f"Rank variance {rank_variance} out of bounds [{lower_bound}, {upper_bound}]"
    }

return {
    "metric_name": "Kähler Manifold Volume",
    "metric_value": rank_variance,
    "instances_tested": 1,
    "n_max": n_bits,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank variance out of bounds\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Run the test again with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13822
2	preregistration	ollama_remote	glm4:latest	0	0	9982
3	novelty	ollama_remote	glm4:latest	0	0	9494
4	novelty	ollama_remote	glm4:latest	0	0	8651
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21168
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7666
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7399
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10652
9	judge	ollama_remote	glm4:latest	0	0	27576

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 116410 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/32d29fb2f820.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/32d29fb2f820.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/32d29fb2f820.tar.gz* (if generated)